RecoGenie - AI-Powered Recommendation Engine

Author: Akhilesh Avinash Patil

1. Project Overview & Introduction

RecoGenie, powered by the conversational AI agent Recommendation Engine, is a personalized movie recommendation platform. Designed to go beyond conventional static search bars, RecoGenie leverages Natural Language Processing (NLP) to understand user preferences, histories, and intents.

Key Features

- **Interactive Conversational Agent**: Users can chat with the AI to ask about specific genres, directors, or general movie trends.
- **Dynamic Preference Tracking**: User profiles are maintained, learning from their chat history and explicit preference updates.
- **Hybrid Recommendation Engine**: The backend merges Collaborative Filtering (finding similar users) and Content-Based Filtering (finding similar movies) to deliver highly accurate suggestions.
- **Modern Interactive UI**: A clean, single-page application experience where users can view recommendations in real-time alongside their chat.

Target Audience

Movie enthusiasts who want tailored film suggestions based on complex, conversational queries rather than rigid filters.

---

2. System Architecture & Tech Stack

The platform is designed with a modern, decoupled architecture:

- **Frontend Layer (Client)**
  - **Technologies**: HTML5, Vanilla CSS, JavaScript (app.js).
  - **Responsibilities**: Managing DOM updates for the chat interface, handling User Profile data fetching, and rendering movie recommendation cards seamlessly via REST API calls to the server.
- **Backend API Layer (Server)**
  - **Technologies**: Python, **FastAPI** (routes.py), Pydantic, Uvicorn.
  - **Responsibilities**: Exposing RESTful endpoints (/chat, /profile, /recommend), data validation, error handling, and orchestrating requests to the AI agents and Data logic.
- **AI/LLM Layer (Brain)**
  - **Technologies**: **LangChain**, HuggingFace (or Groq), FAISS (Vector Store).
  - **Responsibilities**: Processing incoming user queries (agents/conversation_agent.py), retrieving contextual prompts (llm/prompts.py), and generating human-like, knowledgeable responses.
- **Recommendation Engine Layer (Logic)**
  - **Technologies**: Scikit-Learn, Pandas, Numpy.
  - **Responsibilities**: Processing the ml-latest-small MovieLens dataset (recommendation_engine/). It runs both Collaborative (collaborative.py) and Content-Based (content_based.py) models, merging their scores via the Hybrid Engine (hybrid.py) for the final Top-N movie output.

3. Setup and Installation Guide

Follow these steps to deploy RecoGenie locally:

Prerequisites

1. Python 3.9+ installed on your system.
2. API Keys for your chosen LLM provider (e.g., HuggingFace Token or Groq API Key). Create a .env file in the root directory:

   ```
   HUGGINGFACE_API_KEY=your_api_key_here
   GROQ_API_KEY=your_api_key_here
   ```

Installation Steps

1. **Clone the Repository** and navigate to the project root:

   ```
   cd MovieHub
   ```

2. **Create and Activate a Virtual Environment** (Optional but recommended):

   ```
   python -m venv .venv
   # Windows
   .venv\Scripts\activate
   # macOS/Linux
   source .venv/bin/activate
   ```

3. **Install Dependencies**: Use the provided requirements file:

   ```
   uv add -r requirements.txt
   ```

Execution

1. **Start the Backend Server**: Use Uvicorn to run the FastAPI app.

   ```
   uv run uvicorn main:app --reload
   ```

   The API will be accessible at http://127.0.0.1:8000.

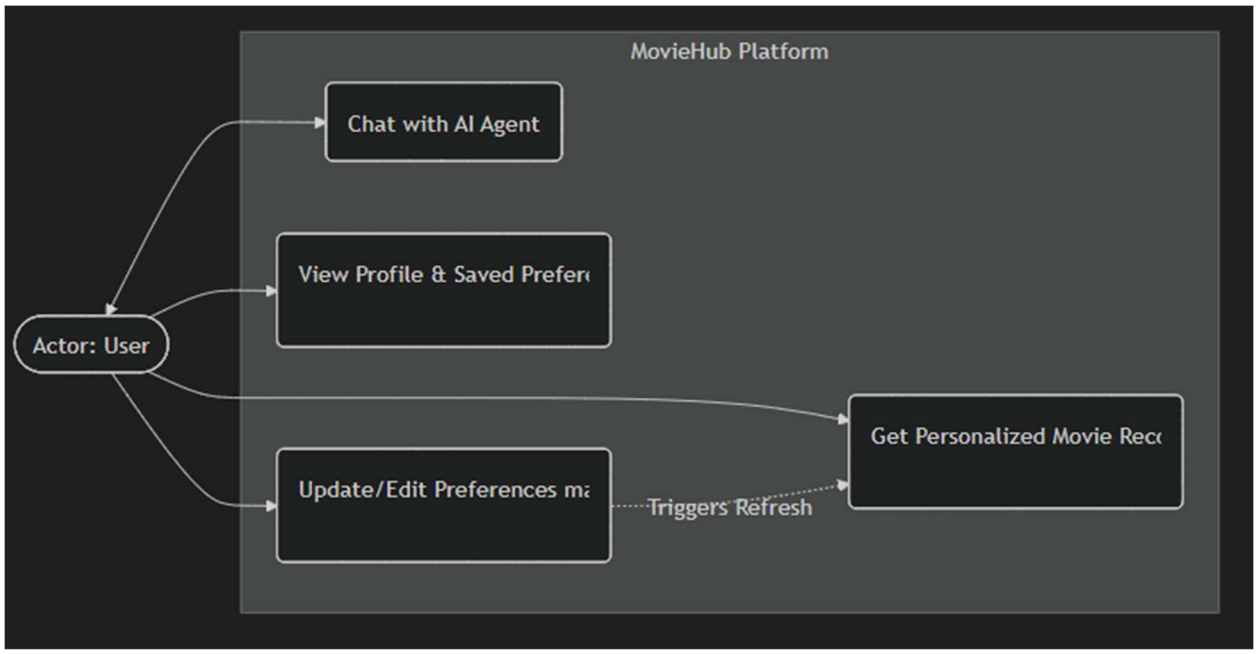
2. **Launch the Frontend**: Open frontend/index.html in your preferred modern web browser.

4. Visualizing System Flow (Diagrams)

To better understand the interaction between the components, refer to the following workflow diagrams.

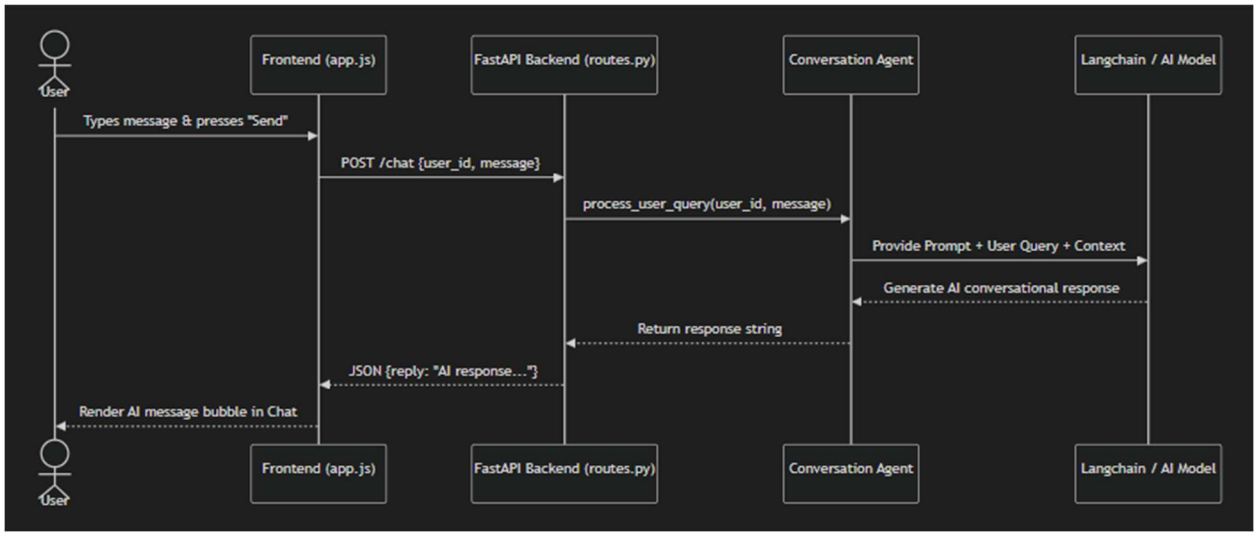
4.1 System Use Case Diagram

This diagram outlines the primary interactions a user has with the RecoGenie platform.



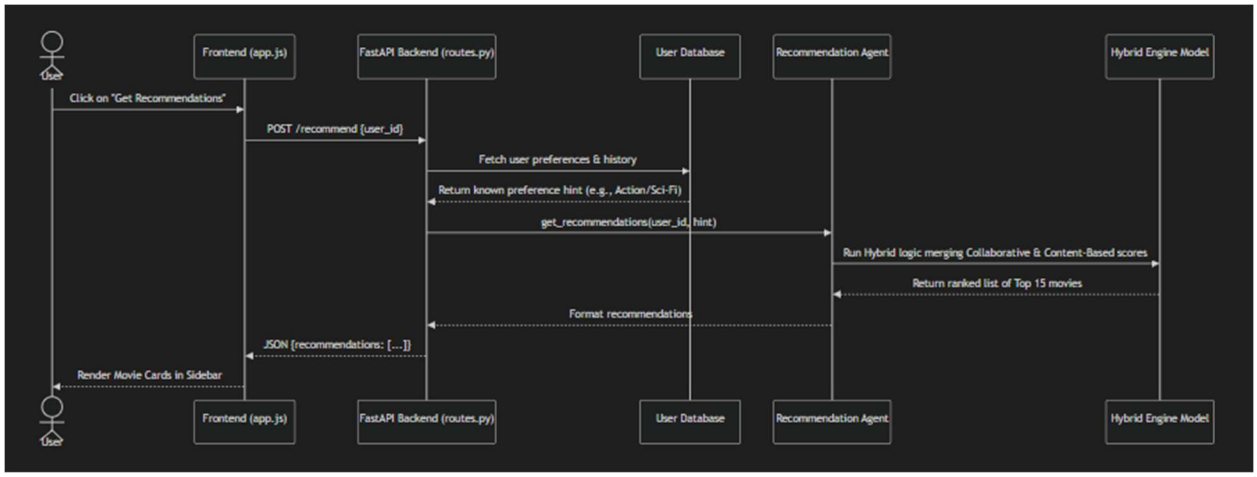
4.2 Chat Agent Workflow (Sequence Diagram)

This sequence diagram visualizes the data flow when a user sends a message through the chat interface.



4.3 Recommendation Engine Workflow (Sequence Diagram)

This sequence diagram shows how RecoGenie retrieves and calculates personalized movie recommendations.


User
Frontend (app.js)
FastAPI Backend (routes.py)
User Database
Recommendation Agent
Hybrid Engine Model
Click on "Get Recommendations"
POST /recommend {user_id}
Fetch user preferences & history
Return known preference hint (e.g., Action/Sci-Fi)
get_recommendations(user_id, hint)
Run Hybrid logic merging Collaborative & Content-Based scores
Return ranked list of Top 15 movies
Format recommendations
JSON {recommendations: [...]}
Render Movie Cards in Sidebar
User
Frontend (app.js)
FastAPI Backend (routes.py)
User Database
Recommendation Agent
Hybrid Engine Model

5. Conclusion

RecoGenie demonstrates the power of combining modern Large Language Models (LLMs) with traditional machine learning recommendation techniques to deliver a hyper-personalized user experience. By shifting the paradigm from static search filters to an intuitive conversational assistant, the platform makes finding movies effortless while actively learning from the user over time. This dual-layer approach—processing natural language for dynamic interaction and crunching hybrid matrices for accurate suggestions—serves as a robust and scalable foundation for next-generation, AI-driven applications.